

How Do Loops Work?

As you learned in earlier modules, loops are used to repeat a block of code for a set number of times. In this lesson you will learn how to work with different types of loops in Python.

The first loop we will go over is the `for` loop. Here is an example of using a `for` loop to iterate through a list and print each item to the console:

```
programming_languages = ['Rust', 'Java', 'Python', 'C++']

for language in programming_languages:
    print(language)
```

The result would be:

```
Rust
Java
Python
C++
```

Notice that the `print(language)` is indented inside of the loop. Without that indentation, you would get an `IndentationError`:

```
"""
Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
IndentationError: expected an indented block after 'for'
statement on line 3
"""
```

You can also use a `for` loop to iterate through other iterables like a string. Here is an example of using a `for` loop to loop through the string `code` and print out each character:

```
for char in 'code':
```

```
print(char)
```

The result would be:

```
c  
o  
d  
e
```

Just like in JavaScript, you can also nest `for` loops in Python. Here is an example of using a nested `for` loop:

```
categories = ['Fruit', 'Vegetable']  
foods = ['Apple', 'Carrot', 'Banana']  
  
for category in categories:  
    for food in foods:  
        print(category, food)
```

The outer loop will iterate through each `category` in the `categories` list. For each `category`, the inner loop will iterate through each `food` in the `foods` list. Here is the result that will be printed to the console:

```
Fruit Apple  
Fruit Carrot  
Fruit Banana  
Vegetable Apple  
Vegetable Carrot  
Vegetable Banana
```

Another type of loop you can use in Python is the `while` loop. This type of loop will repeat a block of code until the condition is `False`. Here is an example of using a `while` loop for a guessing game:

```
secret_number = 3
```

```

guess = 0

while guess != secret_number:
    guess = int(input('Guess the number (1-5): '))
    if guess != secret_number:
        print('Wrong! Try again.')

print('You got it!')

```

In this example we have a `secret_number` variable with the value of `3` and an initial guess of `0`. Then we use the `input` function to get input from the user, then convert the input string into an integer with the `int()` function, and assign it to the `guess` variable. If the user guesses correctly by inputting `3`, the `while` loop is broken out of and the message `You got it!` is printed to the console. Otherwise, the message `Wrong! Try again.` is printed to the console, and the loop repeats, prompting the user to guess again.

Here's an example result:

```

Guess the number (1-5): 2
Wrong! Try again.
Guess the number (1-5): 1
Wrong! Try again.
Guess the number (1-5): 3
You got it!

```

Just like in JavaScript, Python supports the `break` and `continue` statements.

The `break` statement is used to stop the execution of a loop. Here is an example of using the `break` statement for a list of `developer_names`:

```

developer_names = ['Jess', 'Naomi', 'Tom']

for developer in developer_names:

```

```
if developer == 'Naomi':  
    break  
  
print(developer)
```

In this example, we iterate through a list of `developer_names` and print each name to the console. If the name is equal to `Naomi`, then we break out of the loop. This results in only the name `Jess` being printed to the console.

The `continue` statement is used to skip the current iteration of a loop and move onto the next iteration. Let's modify the example from earlier to use the `continue` statement instead of `break`:

```
developer_names = ['Jess', 'Naomi', 'Tom']  
  
for developer in developer_names:  
    if developer == 'Naomi':  
        continue  
    print(developer)
```

Now the result in the console will be different. The names `Jess` and `Tom` are printed because the `continue` statement skips the second iteration of the loop when `developer` is equal to `Naomi`, and does not print that name to the console.

Both `for` and `while` loops can be combined with an `else` clause, which is executed only when the loop is not terminated by a `break` statement. Here is an example of using multiple `for` loops:

```
words = ['sky', 'apple', 'rhythm', 'fly', 'orange']  
  
for word in words:  
    for letter in word:  
        if letter.lower() in 'aeiou':  
            print(f'{word} contains the vowel {letter}')  
            break
```

```
else:
```

```
    print(f'{word}' has no vowels")
```

In this example we have a list of random words, and a `for` loop is used to loop through each word. Inside the outer `for` loop, we have another `for` loop to loop through each letter of each word. If the lowercase version of the letter is a vowel, we print the word followed by what vowels it contains, then break out of the inner loop. If the word contains no vowels, then we print a message indicating that.

Here is what the result looks like in the console:

```
'sky' has no vowels
```

```
'apple' contains the vowel 'a'
```

```
'rhythm' has no vowels
```

```
'fly' has no vowels
```

```
'orange' contains the vowel 'o'
```

Loops are very common in Python, so it's important to get comfortable with them. In the next few lessons, you will learn how to work with the `enumerate()` and `range()` functions in loops.

What Are Ranges and How Can You Use Them in a Loop?

The `range()` function is used to generate a sequence of integers. Here is the basic syntax for the `range()` function:

```
range(start, stop, step)
```

The required `stop` argument is an integer that represents the end point for the sequence of numbers being generated. Here is an example of using the `range()` function:

```
for num in range(3):
```

```
    print(num)
```

The following code generates a sequence of numbers between 0 and 2. The integer 3 is not included because the stop argument is non-inclusive.

If a start argument is not specified, then the default is 0. Otherwise, you can use the optional start argument to start the sequence of integers at a integer other than 0. Here is an example of generating a sequence of integers between 1 and 4:

```
for num in range(1, 5):  
    print(num)
```

By default the sequence of integers will increment by 1. But if you want to change that default, then you can use the optional step argument. Here is an example of generating a sequence of even integers between 2 and 10:

```
for num in range(2, 11, 2):  
    print(num)
```

As mentioned earlier, there is only one required argument for the range() function. If you don't provide any arguments to range(), then you'll get a TypeError:

ERROR!

Traceback (most recent call last):

File "<main.py>", line 1, in <module>

TypeError: range expected at least 1 argument, got 0

It is important to note that the range() function only accepts integers as arguments, not floats. Remember that floats are numbers with decimal points like 3.4. If you try to pass floats as arguments, you'll get a TypeError:

ERROR!

Traceback (most recent call last):

File "<main.py>", line 1, in <module>

TypeError: 'float' object cannot be interpreted as an integer

If you want to generate a sequence of integers in decrementing order, then you can use a negative integer for the `step` argument, like this:

```
for num in range(40, 0, -10):  
    print(num)
```

The following code prints the numbers `40`, `30`, `20`, and `10` in that order to the console.

Another thing you can do with the `range()` function is create a list of integers by using it with the `list` constructor. The `list` constructor is used to convert an iterable into a list. Here is an example of generating a list of even integers between `2` and `10`:

```
numbers = list(range(2, 11, 2))  
print(numbers) # [2, 4, 6, 8, 10]
```

The `range()` function is a very handy way to generate a sequence of integers in Python. Once you get the hang of it, you'll probably find yourself using it a lot in your Python programs.

What Are the Enumerate and Zip Functions and How Do They Work?

In previous lessons you learned how to work with the `for` loop, which is used to repeat a block of code a set number of times. Here is an example of using a `for` loop to print each language from the `languages` list to the console:

```
languages = ['Spanish', 'English', 'Russian', 'Chinese']  
  
for language in languages:  
    print(language)
```

But what if you wanted to keep track of the index for each element? Well, one option is to create an `index` variable and increment it by `1` for each iteration of the loop, like this:

```
languages = ['Spanish', 'English', 'Russian', 'Chinese']
```

```
index = 0
```

```
for language in languages:
```

```
    print(f'Index {index} and language {language}')
```

```
    index += 1
```

While that works, an easier way to do that is by using the `enumerate()` function. The `enumerate()` function keeps track of the index for an iterable and returns an enumerate object.

If we pass the `languages` list to the `enumerate()` function and convert its returned value into a list with the `list()` function, it looks like this:

```
languages = ['Spanish', 'English', 'Russian', 'Chinese']
```

```
list(enumerate(languages))
```

```
# [(0, 'Spanish'), (1, 'English'), (2, 'Russian'), (3, 'Chinese')]
```

Each entry in the enumerate object (now a list) is a tuple containing a count, followed by a value from the iterable passed to the `enumerate()` function.

Now, let's refactor the example from earlier to use the `enumerate()` function:

```
languages = ['Spanish', 'English', 'Russian', 'Chinese']
```

```
for index, language in enumerate(languages):
```

```
    print(f'Index {index} and language {language}')
```

We unpack the count and value for each tuple in the enumerate object into variables named `index` and `language`, respectively. Finally, both those variables are used in an f-string that's printed to the console in each iteration of the loop.

Index 0 and language Spanish

Index 1 and language English

Index 2 and language Russian

Index 3 and language Chinese

This removes the need for manually creating and updating an `index` variable.

The `enumerate()` function also accepts an optional `start` argument that specifies the starting value for the count. If this argument is omitted, then the count will begin at `0`. Here is an example of using the optional `start` argument:

```
languages = ['Spanish', 'English', 'Russian', 'Chinese']
```

```
for index, language in enumerate(languages, 1):  
    print(f'Index {index} and language {language}')
```

And here is what the result will look like in the console:

Index 1 and language Spanish

Index 2 and language English

Index 3 and language Russian

Index 4 and language Chinese

So far we've only been iterating over one list. But what if you need to iterate over multiple iterables in parallel? Well, you can use the `zip()` function for that, which combines lists into pairs of elements and returns an iterator of tuples.

If we pass a list of `developers` and `ids` to the `zip()` function and convert its returned value into a list with the `list()` function, here's what it looks like:

```
developers = ['Naomi', 'Dario', 'Jessica', 'Tom']
```

```
ids = [1, 2, 3, 4]
```

```
list(zip(developers, ids))
```

```
# [('Naomi', 1), ('Dario', 2), ('Jessica', 3), ('Tom', 4)]
```

And here's an example of using the `zip()` function with a `for` loop to iterate over `developers` and `ids`:

```
developers = ['Naomi', 'Dario', 'Jessica', 'Tom']
ids = [1, 2, 3, 4]

for name, id in zip(developers, ids):
    print(f'Name: {name}')
    print(f'ID: {id}')
```

In this example, `zip()` combines the two lists into pairs of elements and returns an iterator of tuples. The `for` loop then unpacks each tuple into `name` and `id`. Finally, for each print statement, we are printing each `name` and `id` from the `ids` and `developers` lists respectively. Here is what the result looks like in the console:

```
Name: Naomi
ID: 1
Name: Dario
ID: 2
Name: Jessica
ID: 3
Name: Tom
ID: 4
```

The `enumerate()` and `zip()` functions are very powerful, and when combined with loops, can make your code much more concise.